

The Agent Stack - Part 8: Observability, Evaluation, and Production Feedback Loops

Why demos are stories, but production needs evidence



VINOTH GOVINDARAJAN

MAY 18, 2026



9



2



2

Share

A demo is allowed to be a story.

Production is not.

At 3:07 AM, the question is not whether the agent looked smart in the demo. The question is why a customer received the wrong refund, which run did it, what context it used, which tool acted, whether approval was required, and whether the next release would catch the same failure.

That is what this final part is about.

Observability preserves the run. Evaluation judges the behavior. A feedback loop decides what changes before the next release.

My read is that this layer is not observability alone.

It is the evidence loop.

A demo is a story. Production is a reconstruction problem.

Most agent demos end when the answer looks right.

The user asks. The agent reasons. A tool runs. The final response sounds useful.

That is a valid demo. It proves one path can work.

Production asks a different question.

Can you reconstruct the path when it does not work?

The transcript might show the user's request and the agent's final answer. That is not enough. In an agent system, the final answer is only the last visible step in a longer chain.

The system may have loaded a session, retrieved documents, pulled memory, exposed tools, applied policy, requested approval, and changed something outside the transcript.

If something goes wrong, the operator needs to know more than what the agent said.

They need to know what the system saw, what it owned, what it exposed, what it allowed, what it did, and what changed.

That is why production agents need evidence.

Not just logs.

Not just dashboards.

Not just eval scores.

Evidence.

A demo can be narrated from the happy path. Production has to be reconstructed from the artifacts the system leaves behind.

Observability gives you those artifacts: traces, logs, metrics, transcripts, context snapshots, state snapshots, health signals, and audit records.

For agent systems, that evidence has to cover more than service behavior. It has to cover model inputs, context assembly, retrieval, memory, tools, approvals, and side effects.

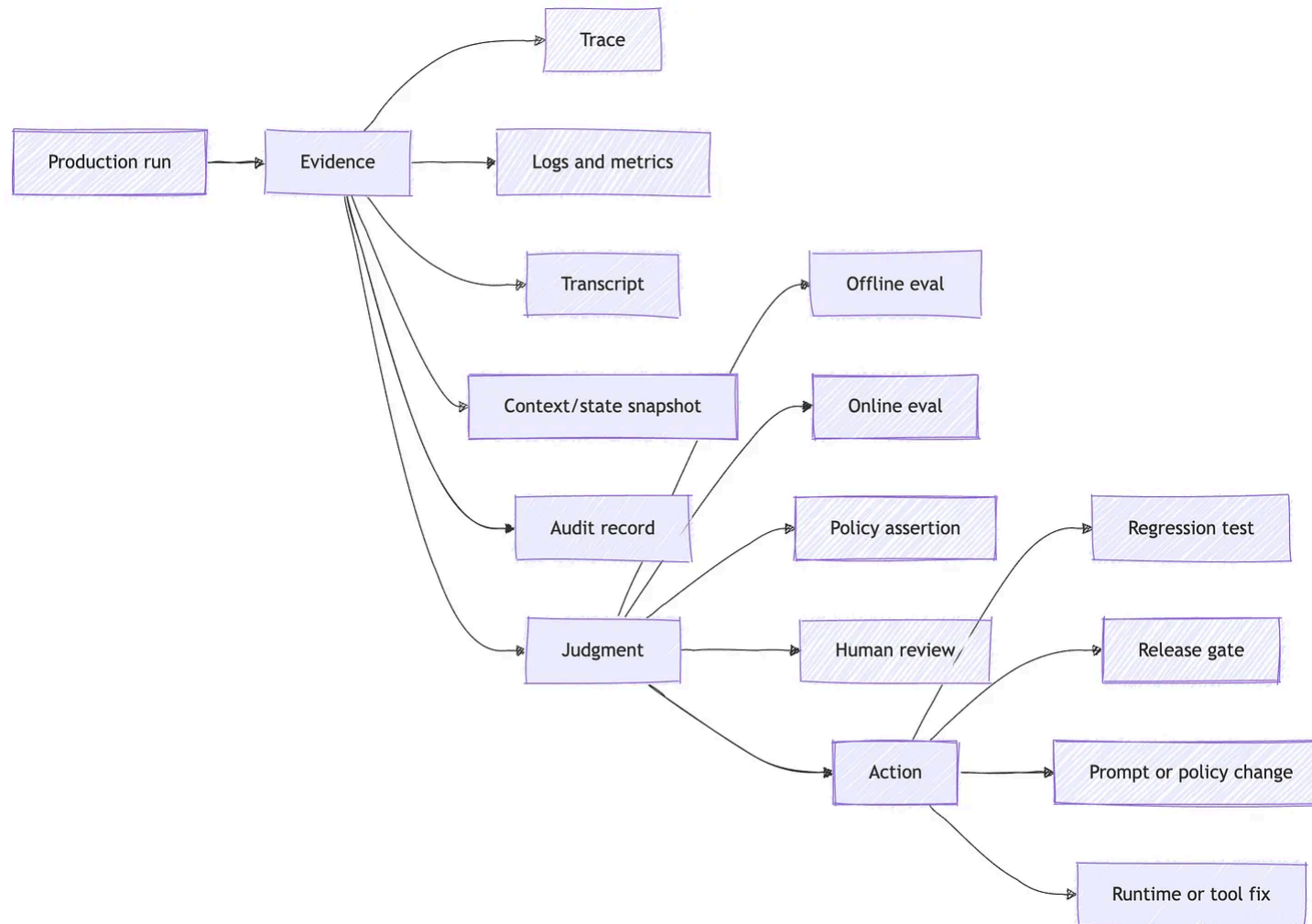
That is the first boundary to keep clear.

Observability is not the answer to “was this good?”

Observability is the answer to “what happened?”

A trace is not a verdict. It explains what happened. It does not decide whether the behavior was acceptable.

If you remember one picture from this post, make it this one.



The evidence has to cross the whole stack.

A useful agent trace should not stop at the model call.

That is the common mistake.

A team instruments the LLM request and response, then calls the system observable. But the model call is only one part of the run.

For a production agent, the interesting path crosses the stack.

An event enters through a channel. The control plane assigns the run and session. The runtime assembles context. The model produces output. The runtime validates whether a tool is available. A tool call crosses into an execution surface. A policy or approval boundary may apply. A side effect may happen. The response returns to the user.

If the trace only captures the model request, it misses the system.

A run is not one operation. It is a chain of operations with causality between them.

For agent systems, my read is that the trace should preserve the chain of custody for a decision.

At minimum, for meaningful runs, I would want to reconstruct:

- the run ID and session ID
- the user-visible event that started the run

- the model and prompt version
- the context assembled for the turn
- the retrieved documents or memories used
- the tools exposed to the model
- the tool calls requested
- the validation and policy decisions
- the approval decision, if one was needed
- the execution result
- the side effect, if the tool changed something
- the final response
- the relevant versions of prompts, tools, policies, retrieval configs, and memory rules

Some of that looks like normal distributed tracing.

Some of it is agent-specific.

The exact schema is still moving. That is fine. Production systems should not depend on one vendor's object model becoming the universal shape of agent observability.

The invariant is simpler than the schema.

Every important run should leave enough evidence to explain how the system reached the result.

If it acted, the evidence should also explain under what authority.

That brings us to audit.

A trace is not an audit trail.

A trace explains execution.

An audit trail preserves accountability. *What accountability is?*

Those are related, but they are not the same responsibility.

A trace asks:

- Where did the run go?
- Which spans executed?
- Where did time go?
- Which tool was called?

- Which error happened?

An audit trail asks:

- Who or what acted?
- Which identity was used?
- What scope was granted?
- Which policy applied?
- Was approval required?
- Who approved it?
- What object changed?
- What was the result?

That distinction matters once an agent crosses from chat into action.

If an agent drafts a reply, a trace may be enough.

If an agent sends the reply, changes a permission, issues a refund, updates a customer record, writes to a filesystem, or deploys code, debugging evidence is not enough. The system needs an accountability record.

My read is that traces and audit records should share identifiers, but not responsibilities.

The trace can show that the runtime called the refund tool after retrieving the policy document.

The audit record should show which user or service identity authorized the refund, which account was affected, what amount changed, whether approval was required, and which run caused the action.

Those records can live near each other. They can point to each other.

But they should not collapse into one vague “logs” bucket.

When teams collapse them, two bad things happen.

The debug stream becomes too sensitive and noisy for broad engineering use.

The audit record becomes too weak for accountability.

Agents make this boundary more important because the same user-visible answer can hide very different authority paths underneath it.

The model may suggest.

The runtime may decide.

The policy layer may allow.

The execution surface may act.

The audit trail has to preserve that distinction.

Evaluation is judgment, not telemetry.

Once you can reconstruct the run, the next question is whether the behavior was acceptable.

That is evaluation.

This is where many teams blur the boundary.

They add tracing and think they have evals.

Or they add an LLM judge and think they have observability.

Neither is enough.

A trace can describe a bad run perfectly. It can show every model call, every retrieved chunk, every tool request, every approval decision, and every response. The trace still

does not decide whether the behavior met the bar.

Evaluation applies judgment to the evidence.

For agent systems, that judgment cannot stop at the final answer.

A final answer can sound good while the agent used the wrong source, skipped a policy check, selected the wrong tool, passed the wrong argument, or changed the wrong object.

So the eval target has to match the system.

For a writing assistant, final response quality may matter most.

For a support agent, policy adherence and escalation behavior may matter.

For a refund agent, final-state correctness matters.

For a database agent, query safety and authorization matter.

For a retrieval-heavy agent, evidence quality and groundedness matter.

For a long-running workflow, resumability and step correctness matter.

This is why “agent evals” are not one thing.

Some checks should be deterministic. Did the agent call the right tool? Did it pass the right account ID? Did it stay under the dollar limit? Did approval happen before the side effect?

Some checks need a rubric. Was the answer clear? Did it explain uncertainty? Did it use retrieved evidence faithfully?

Some checks need human review. Was the policy interpretation correct? Was the escalation appropriate? Was the user outcome acceptable?

That variety matters.

Do not ask a model judge to prove something your system can assert. Still evals? or unit tests?

If a tool call should never happen without approval, write an assertion.

If the final database state should match a goal state, check the state.

If the response quality is subjective, use a rubric and calibrate it.

Evaluation is not one magic score.

It is a set of judgments attached to the parts of the run that matter.

Feedback is not learning until the system changes.

This is the most important boundary in the whole post.

Feedback is not learning.

A thumbs-down event is not learning.

A human annotation is not learning.

A bad production trace is not learning.

An online eval failure is not learning.

Those are signals.

The system only improves when a process turns those signals into a change.

That change might be a new regression test. A dataset item. A prompt change. A tool schema change. A retrieval fix. A memory policy update. A routing rule. An approval threshold. A sandbox restriction. A runtime bug fix. A release gate.

Without that path, feedback is just a labeled observation.

This is where traces become useful beyond debugging.

A bad trace should be triaged. The failure should be labeled. The example should become a dataset item. The dataset item should become a regression. The regression should run before release. The release should be blocked if the known failure comes back.

That is production learning.

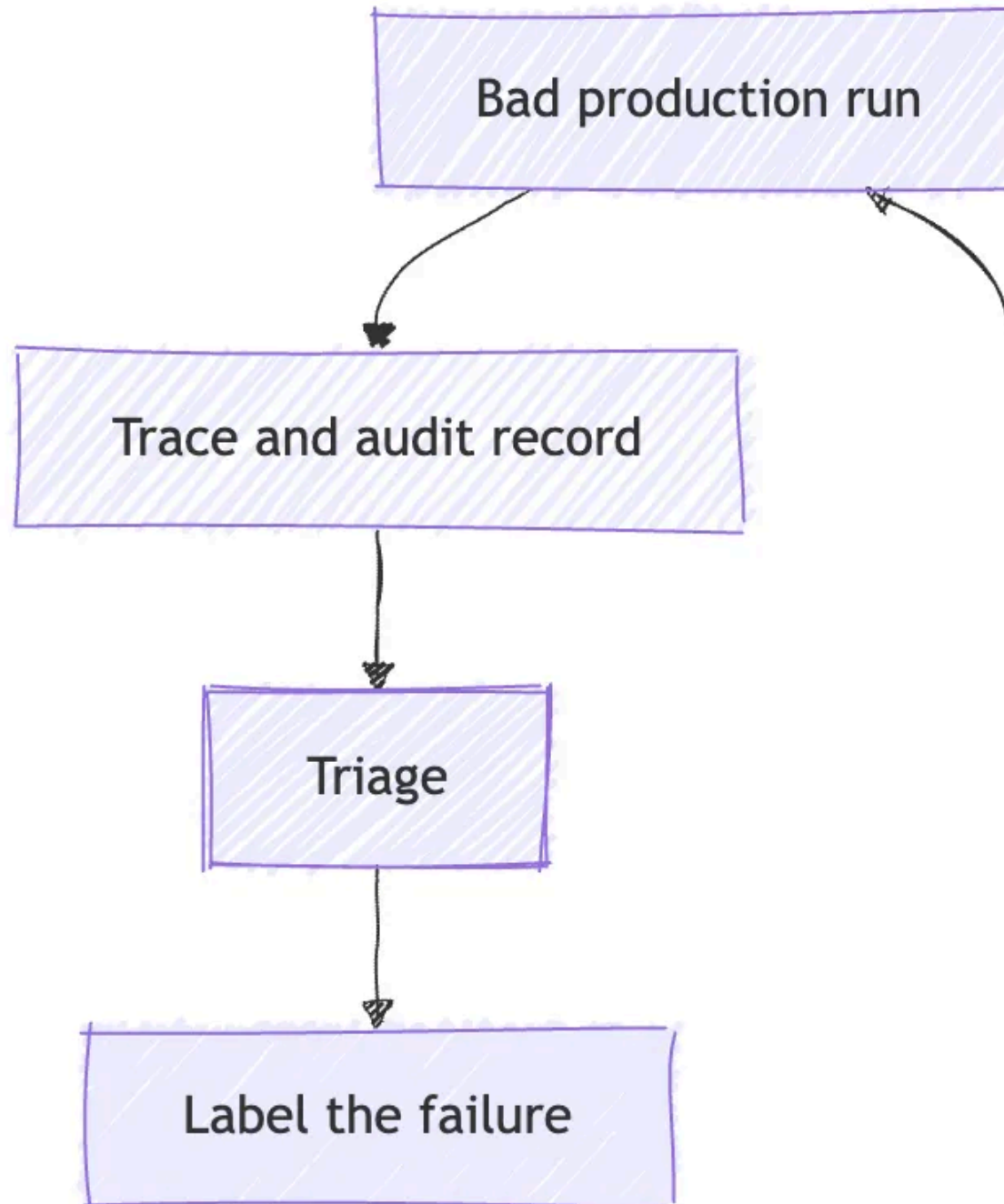
Not because the model automatically learned.

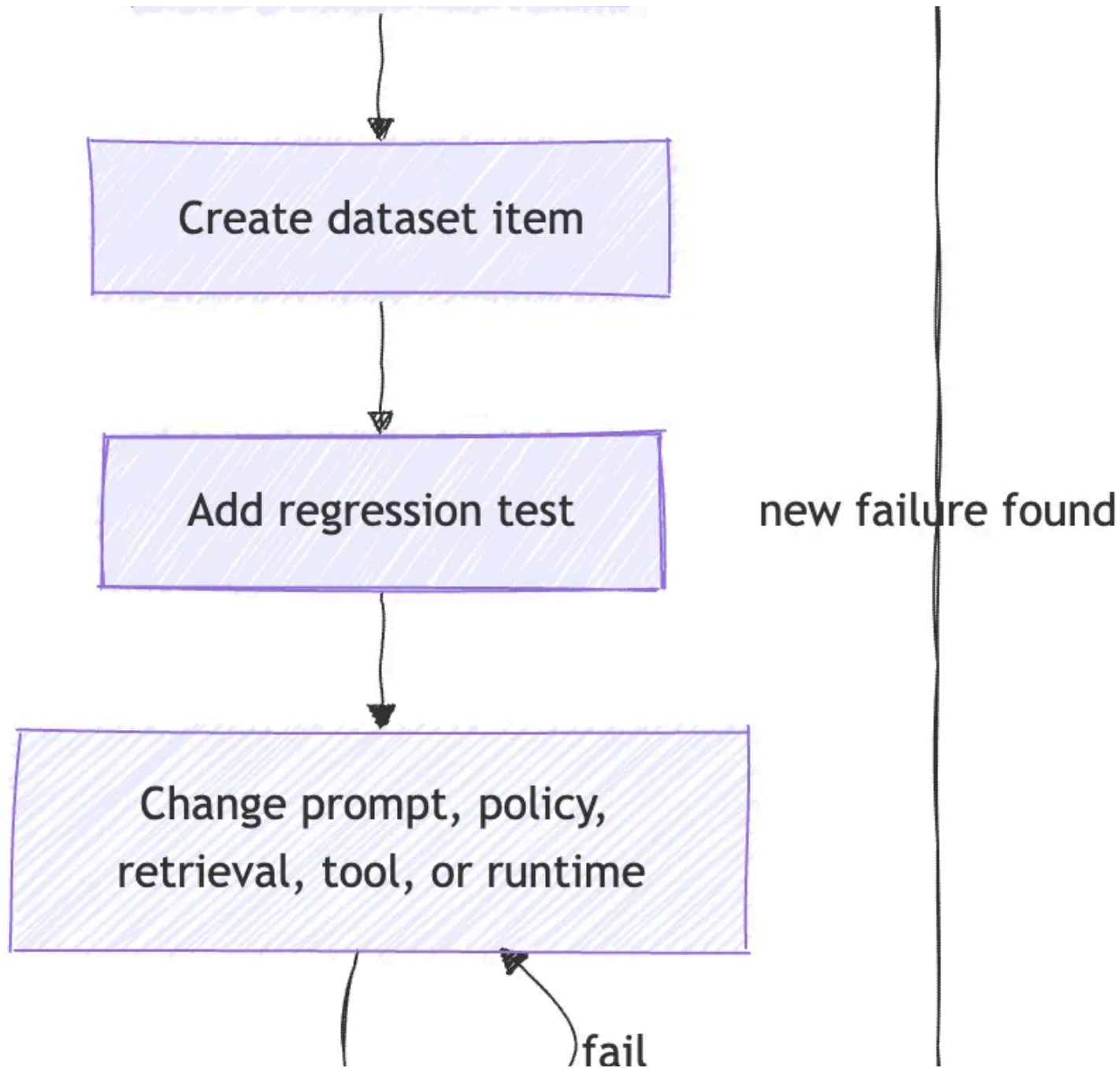
Not because a dashboard exists.

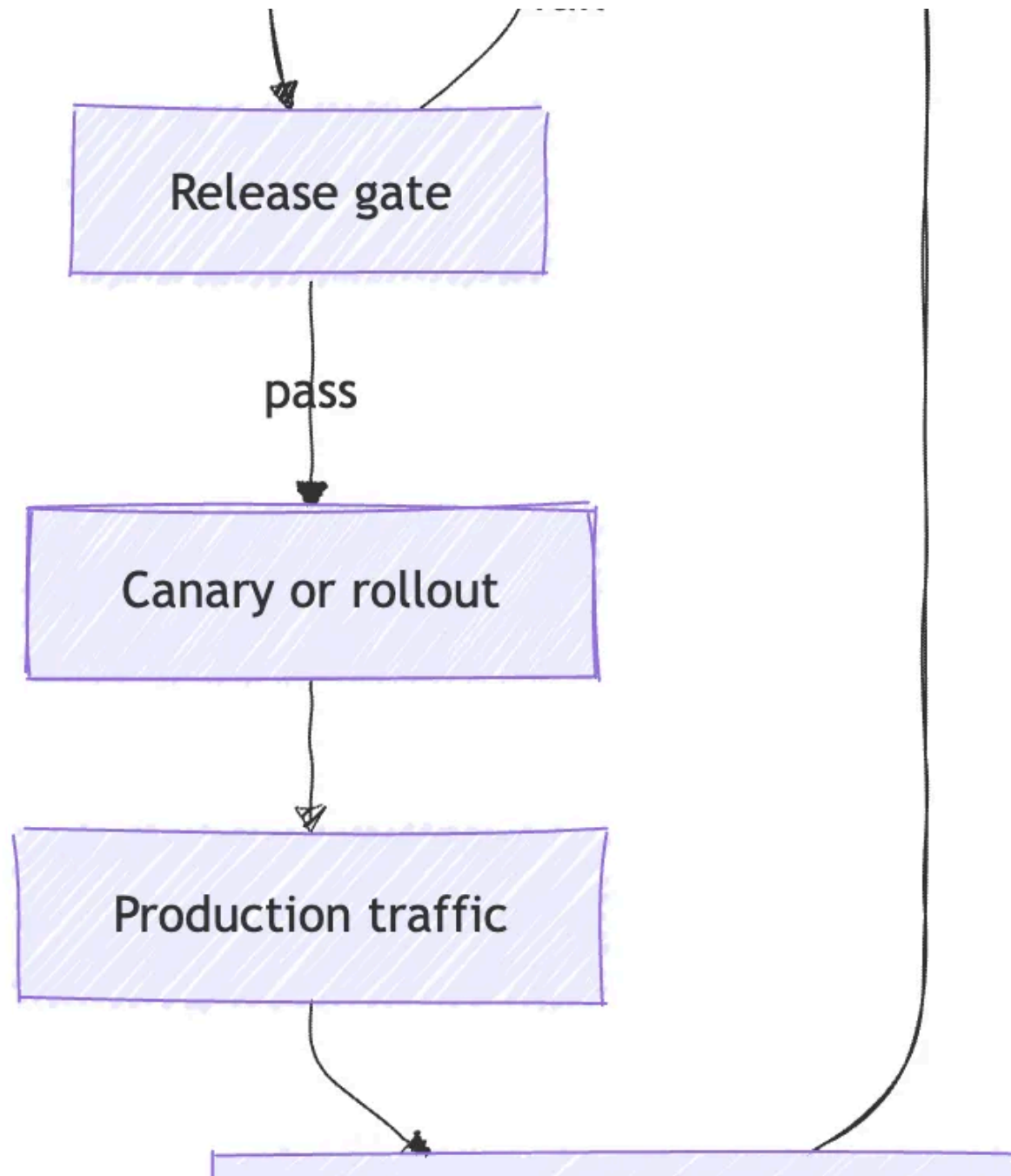
Because the organization turned observed behavior into a system change.

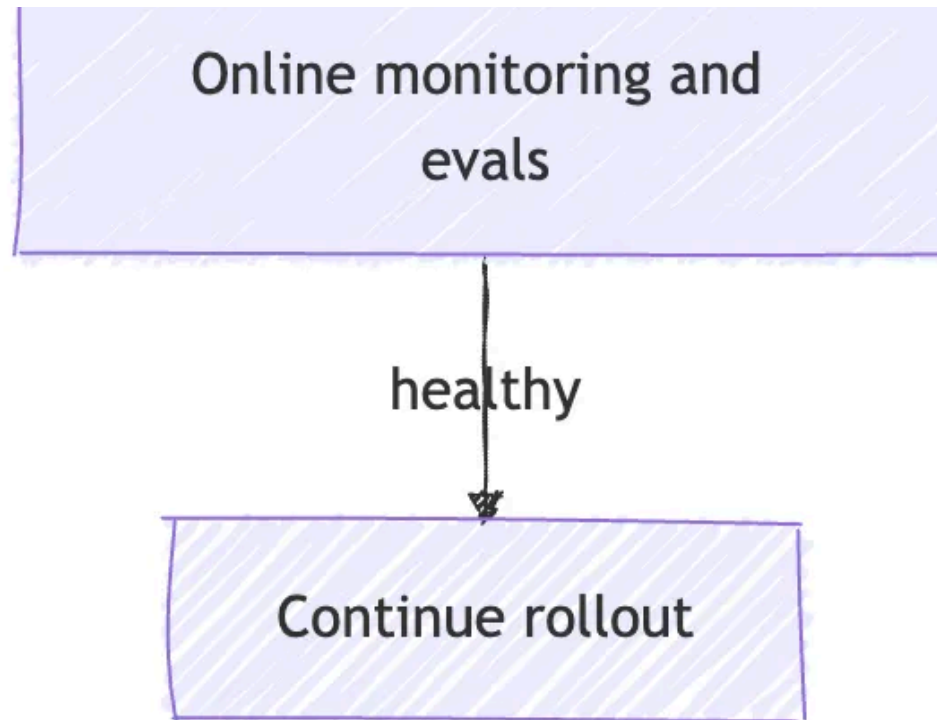
Feedback is not learning until some part of the system changes.

The loop looks like this:









The important box is not the dashboard.

It is the release gate.

A dashboard informs someone.

A gate changes what ships.

Agent systems need that discipline.

A risky workflow should have release criteria.

For example:

The agent must choose the correct tool in these cases.

The agent must not call this tool without approval.

The agent must retrieve the policy document for this class of question.

The agent must refuse this action under these scopes.

The agent must complete the task within this latency and cost budget.

The agent must preserve final-state correctness in these workflows.

The agent must not regress on known production failures.

That last line is the important one.

Known failures should become harder to repeat.

| About this section, which parts should be automated processes? Is it related with CI/CD?

Failure modes

Treating traces as proof of quality

A trace can show exactly what happened and still describe a bad run.

Tracing gives evidence. It does not give judgment.

Evaluating only the final answer

The final answer is the most visible artifact, but it is not the whole task.

For agents that act, the path matters. Tool choice matters. Arguments matter.
Approval matters. Final state matters.

Losing session identity

Many agent failures happen across turns.

A stale memory. A wrong session. A summarized transcript that dropped a constraint.
A workflow resumed under the wrong context.

If traces are not tied to session identity, the operator sees fragments instead of the run.

Burying audit records inside debug logs

Debug logs are optimized for engineering diagnosis.

Audit records are optimized for accountability.

If the system changes the outside world, keep those responsibilities separate.

Collecting feedback with no path to action

User feedback, annotations, and eval failures are useful only if they can change something.

A label with no owner is not a loop.

Building dashboards that never gate releases

A dashboard can tell you the system is worse.

A gate can stop the worse version from shipping.

Production systems usually need both.

Over-trusting model graders

Model graders are useful for fuzzy judgment.

They are not a replacement for deterministic checks, policy assertions, human review, or final-state validation.

Creating an observability privacy problem

Agent traces can contain prompts, responses, retrieved documents, tool outputs, user identifiers, and application state.

That means observability needs retention, access control, redaction, and sampling rules.

More evidence is not always better if the evidence becomes a new sensitive data store.

Builder checklist

If I were reviewing this layer in a production agent system, I would ask for these invariants:

1. **Every meaningful run has a trace ID and session ID.**

A user-visible outcome should map back to the run that produced it.

2. **The trace crosses the stack.**

It should cover context assembly, model calls, tool exposure, tool calls, policy checks, approvals, execution results, and final response.

3. **Every side effect has an audit record.**

Trace the execution. Audit the authority.

4. **Important versions are reconstructable.**

Track prompt, model, tool schema, policy, retrieval config, memory rule, and

runtime version.

5. Evaluation targets the path and the outcome.

Check tool correctness, approval behavior, policy adherence, retrieval quality, and final state, not just final answer quality.

6. Known failures become regressions.

A serious failure is not closed until the system can detect that class of failure again.

7. High-risk releases have gates.

Use evals, SLOs, policy assertions, latency, cost, and known-regression coverage to decide what ships.

8. Feedback has an owner.

User feedback, human review, online evals, and incidents need a path into datasets, tests, prompts, policies, tools, or runtime changes.

Recap

This layer is easy to flatten into monitoring.

That misses the point.

Observability preserves evidence.

Evaluation applies judgment.

Audit records preserve accountability.

Feedback loops change the system.

Release gates make the loop real.

The goal is not to measure everything.

The goal is to preserve the evidence that matters, judge it against the right bar, and make known failures harder to repeat.

Closing the series

This is where the stack comes back together.

Interfaces create events.

The control plane decides what the run is.

The runtime decides how the run proceeds.

The model engine generates output inside the context it was given.

Context, retrieval, and memory shape what the model sees.

Tools expose capabilities.

Execution surfaces turn tool calls into side effects.

Identity, policy, and approvals decide what is allowed.

Observability, evaluation, and feedback loops decide whether the system can explain itself and improve.

That is the difference between a demo and an operated system.

A demo shows that an agent can complete one story.

Production asks whether the system can reconstruct the story when it goes wrong, judge the behavior, and change what ships next.

If there is one sentence to keep from this series, it is this:

model output becomes bounded action only when the surrounding system owns the boundaries.

Part 8 is about how those boundaries get better in production.

Not automatically.

Not magically.

Through evidence, judgment, and system change.

References

- [OpenTelemetry Observability Primer](#), [Traces](#), [Logs](#), and [GenAI Semantic Conventions](#)
- [OpenAI Agents SDK Tracing](#), [Evaluate agent workflows](#), [Evals](#), [Graders](#), and [Data controls](#)
- [LangSmith Observability Concepts](#), [Evaluation Concepts](#), and [Automation Rules](#)
- [Langfuse Observability Data Model](#), [Sessions](#), and [Datasets](#)
- [Braintrust Evaluate](#), [Observe](#), and [Datasets](#)
- [NIST Glossary: Audit Trail](#), [Google SRE: Implementing SLOs](#), [Canarying Releases](#), [Postmortem Culture](#), and [Monitoring Distributed Systems](#)

The Agent Stack v1

Thanks for reading The Agent Stack. I write about the systems layer behind modern AI: control planes, runtimes, context, memory, tools, execution surfaces, trust boundaries, and production reliability. Subscribe to follow the next set of essays.

Subscribe

- [Part 1: A Systems Map of Modern Agent Infrastructure](#)
- [Part 2: Foundation, Infrastructure, Models, and Inference](#)
- [Part 3: Control Planes, Sessions, and State Ownership](#)
- [Part 4: Runtimes, Workflows, and Durable Execution](#)
- [Part 5: Context, Retrieval, and Memory](#)
- [Part 6: Tools, MCP, and Capability Surfaces](#)
- [Part 7: Execution Surfaces, Identity, and Approval Boundaries](#)

If this series helped

This closes the first version of The Agent Stack.

If the series helped you reason more clearly about agent systems, share it with someone building beyond the demo stage.

Subscribe to The Agent Stack

By Vinoth Govindarajan · Launched 6 months ago

The Agent Stack is a technical deep dive into how modern AI agents actually work under the hood, from control loops and memory systems to orchestration, evaluation, and production deployment.

Subscribe

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



9 Likes · 2 Restacks

Discussion about this post

Comments

Restacks



Write a comment...



Vrinda Damani 22 jun



The part I keep turning over is the gap between knowing what the system did and knowing whether it was any good. They sound like the same question until you sit with them. And the reminder that feedback isn't learning unless something downstream actually changes hit home, otherwise you have just built a neat record of the same problems.

♡ LIKE 💬 REPLY

📌 SHARE



Fruk 20 may

...

"demos are stories, production needs evidence" — stealing this framing. the hardest failure mode i've hit: the agent passes every eval suite, then dies in production because the input distribution shifted. test set was clean, real traffic was not. question: how do you test for distributional drift before it breaks prod?

♡ LIKE 💬 REPLY

📌 SHARE